GPU Periodicity: the Kernel Joins the Work-Reduction Regime

Matias Saavedra

Friday 10th July, 2026

Binary: branch feat/gpu-periodicity, commit 2abc909 (tag binary-v7-gpu-periodicity), built on top of main (c4744c0, the binary-v6-periodicity lineage — the CPU-side check is in both binaries). Baseline is that main commit itself. Raw data: experiments/23-gpu-periodicity/.

Resumen

This report ports the exact Brent periodicity check of 21-periodicity-check.tex from the CPU diverge() to the CUDA kernel — the variant report 21 explicitly deferred with "expect a loss." Measured on the production spec (100 frames, 1920×1080, diffT $= 0.1$, save $= 1$, 3 alternating reps with a CPU12 control, on AC): the prediction is falsified. Mix-controlled kernel time over the identical 5,608 regions falls 66.2 → 28.3 s (2.34×), GPU-only wall falls 78.2 → 40.1 s (−48.8%), and the production hybrid falls 29.83 → 23.72 s (−20.5%) — the best wall this project has recorded — while all 100 output frames stay byte-identical. The error in the prediction was its cost model: a warp does not pay 32×MAXITER when lanes diverge at an early exit; it pays the slowest lane's detection latency, and spatially adjacent lanes detect their cycles at correlated times. The hybrid self-rebalances a third time (GPU share 18 → 54% of regions), the CPU pool stays the binding resource at 20% less cost, and the GPU thread's own cost structure inverts: kernel occupancy drops from 85% to 45% of the wall, promoting host-side commit — report 20's scanLine lever — to the next target.

1. Motivation: the last executor still grinding interiors

binary-v6-periodicity (report 21) eliminated the interior grind on the CPU only: a Brent-style cycle detector returns MAXITER as soon as the orbit state exactly revisits a saved state, cutting CPU12 by −78.6% and the hybrid by −46.3% with byte-identical output. The GPU kernel was deliberately left untouched, on the strength of two kernel-level measurements from report 16: interior regions run at exactly 100.0% warp execution efficiency, and the kernel is FP64-bound at 85.6% of peak. Report 21's recommendation 6 spelled the prediction out: "A/B a GPU-side periodicity variant separately — expect a loss … per-pixel early-out would introduce divergence into that kernel and add FP64 compare work to the bottleneck pipe."

Two developments made that prediction worth testing rather than trusting. First, the repricing analysis of report 22 (branch feat/igpu-opencl) showed that v6 devalued

every executor whose niche was interior content — the dGPU's marginal worth fell to ∼3.4 CPU-worker equivalents ($\mathcal{S}$5.5) precisely because affinity feeds it the big interior regions that CPUs now skip most of. The dGPU was left as the only executor still grinding every interior pixel to MAXITER: on the v6 hybrid, the GPU thread spends 85% of the entire wall inside kernels (Fig. 3), most of it proving what a cycle detector knows after ∼900 iterations (report 21's measured detection latency on this content).

Second, the prediction's cost model does not survive contact with how warps actually execute an early exit. A lane that detects a cycle and returns does not stall its warp; it sits masked while the remaining lanes iterate. The warp's cost is therefore not 32×MAXITER-or-nothing — it is the maximum of its 32 lanes' detection latencies. The 32 lanes of a 16 × 16-block warp are spatially contiguous pixels of the same interior region, attracted to the same cycle, so their latencies are strongly correlated and the maximum is a small multiple of the mean. Against MAXITER = 10,000 on the deep frames, even a 3× inflation of the ∼900-iteration mean latency wins by an order of magnitude. The real costs are the two extra FP64 compares per iteration on every lane (∼25–30% on the ∼7-FLOP recurrence, and on the bottleneck pipe, exactly as report 21 said) and the forfeited efficiency metric. Whether the latency win covers those costs is a measurement, not a derivation — hence this experiment.

2. The change: the CPU detector, verbatim, in the kernel

Commit `2abc909` adds the same anchor-and-compare block that `fa6d8ac` added to `MandelRegion::diverge()`, with the same doubling refresh schedule (anchor starts at z_0, refreshes at iterations 8, 16, 32, . . .), to the device function in `kernel.cu` (comment abridged here; the full exactness argument is in the source):

Código 1: kernel.cu — the device `diverge()` with the Brent check (commit `2abc909`).

```
__device__ int diverge(double cx, double cy, int MAXITER) {
  int iter = 0;
  double vx = cx, vy = cy, tx, ty;
  // Brent-style periodicity check, mirroring the CPU diverge():
  // an exactly-repeating orbit can never escape, so returning
  // MAXITER on exact FP revisit is exact w.r.t. this kernel's own
  // (FMA-contracted) arithmetic. A detecting lane exits and sits
  // masked; the warp's cost drops to its slowest lane's latency.
  double sx = vx, sy = vy;
  int nextSave = 8;
  while (iter < MAXITER && (vx * vx + vy * vy) < 4) {
    tx = vx * vx - vy * vy + cx;
    ty = 2 * vx * vy + cy;
    vx = tx;
    vy = ty;
    iter++;
    if (vx == sx && vy == sy)
      return MAXITER;
    if (iter == nextSave) {
      sx = vx;
```

```
    sy = vy;
    nextSave *= 2;
   }
  }
  return iter;
}
```

The exactness argument is per-device: the iteration map as compiled for this GPU is deterministic, so an exact state revisit proves the orbit cycles forever and the plain loop would have ground to `MAXITER` and returned the same value; an escaping orbit never exactly repeats, so exterior pixels are untouched. The one new hazard relative to the CPU case is codegen: the guarantee is against the kernel's own FMA-contracted arithmetic, so output identity additionally requires that `nvcc` contract the (textually unchanged) iteration statements identically in both builds. That is an empirical question, and $\mathcal{S}4.4$ answers it: it did.

No other code changes. In particular the BlockingSync probe of report 22 lives only on `feat/igpu-opencl` — one lever per branch, so this A/B measures the periodicity check and nothing else.

3. Methodology

Hardware/workload. i7-9750H (6C/12T) + GTX 1660 Ti Max-Q, production `spec.in` (100 frames, 1920×1080, deep zoom to `maxIter` 10,000), `diffT` = 0.1, `pixT` = 32,768, `quiet` = 1, `save` = 1 — the Fig-11.15 methodology of experiments 21 and 22. AC power.

Configurations. `experiments/23-gpu-periodicity/ab.sh`: three configs × two binaries × three reps, binaries alternated back-to-back inside each config so thermal drift hits both sides equally. `gpuonly` (1 thread, GPU enabled) isolates the kernel change — every region of every class goes through the modified kernel. `hybrid` (12 threads) is the production config. `cpu12` (12 threads, GPU off) is a control — the CPU code is byte-identical between the binaries, so any delta it shows is environment, not code — and the same-batch anchor for the worker-equivalence arithmetic of $\mathcal{S}5.5$. The baseline binary was built from `main` (`c4744c0`) in a clean worktree.

Calibration honesty.

The battery was charging during the sweep (added thermal load) and the sweep started on cold silicon. The control caught both: its per-rep delta reads +6.4% → +1.2% → +0.07% across reps 1–3 — while its pool compute is 408.6 s in both binaries to within 0.1 s — so the batch reaches thermal steady state during rep 2 and the control then converges to the wash it must be. Because `base` runs first in every pair, rep 1's cold bias favours the baseline, i.e. 3-rep means understate the improvement. This report therefore quotes steady-state numbers (reps 2–3) with 3-rep means alongside. Repeatability at steady state is excellent: the two `gp` hybrid reps agree to 1.4 ms. An earlier same-day non-charging batch (`logs/smoke_hybrid.out`) ran ∼1.5 s faster in absolute terms at the same relative delta (hybrid 27.93 → 22.44 s, −19.7%) — consistent with the ∼10% thermal-history band documented in report 22.

4. Results

4.1. End-to-end wall

Tabla 1: Per-rep wall times (s). Rep 1 is the cold-start rep — the control row shows how far from steady state the batch was at each point; read the headline from reps 2–3, where the control is a wash.

config	binary	rep 1	rep 2	rep 3	steady mean (2–3)
gpuonly	base	77.049	78.194	78.235	78.214
gpuonly	gp	44.934	40.003	40.140	40.071
hybrid	base	29.079	29.833	29.824	29.829
hybrid	gp	25.593	23.716	23.714	23.715
cpu12	base	35.057	35.804	35.747	35.775
cpu12	gp	37.285	36.238	35.771	36.004

Tabla 2: Steady-state summary (3-rep means in parentheses). Negative Δ means the GPU-side check helped; the control's +0.6% bounds the environmental noise of the batch.

config	baseline (s)	gpu-periodicity (s)	Δ wall	speed-up
GPU-only (1 thread)	78.21 (77.83)	40.07 (41.69)	−48.8% (−46.4%)	1.95×
dGPU+11CPU	29.83 (29.58)	23.72 (24.34)	−20.5% (−17.7%)	1.26×
CPU12 (control)	35.78 (35.54)	36.00 (36.43)	+0.6% (+2.5%)	—

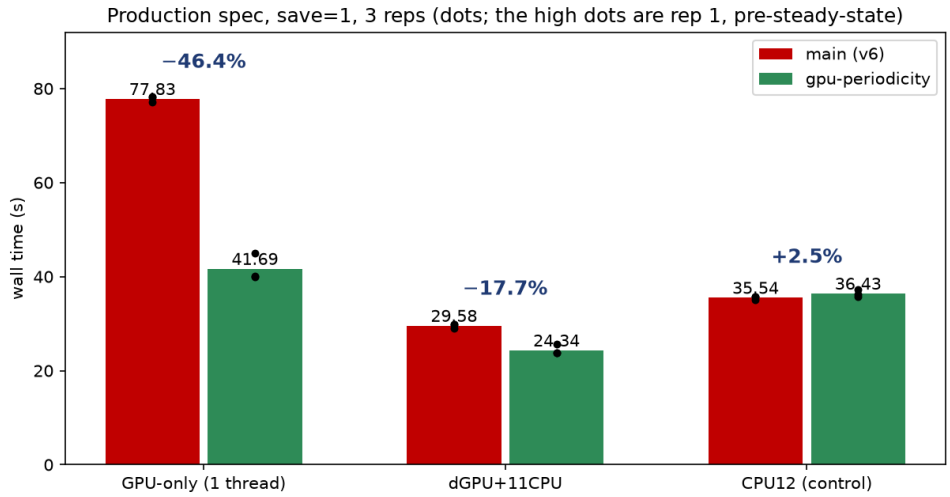

Figura 1: Wall time by configuration; bars are 3-rep means, dots the individual reps. The high green dots are rep 1 (pre-steady-state, biased against the treatment); the control group on the right shows that bias converging to zero. The headline gap is the middle group: the production hybrid at 23.72 s, the best wall recorded for this project.

4.2. Kernel-level, mix-controlled: the gpuonly runs

In the gpuonly config both binaries push the identical 5,608 regions through the kernel, so the CUDA-event totals compare like for like (steady-state means):

Tabla 3: GPU-only lane accounting over the identical 5,608 regions (steady-state means). "Host side" is wall minus kernel minus D→H: per-pixel commit into the QImage, `examine()` sampling, queue. The kernel column is the pure effect of the check; the host column is the part of the lane the check cannot touch.

binary	kernel (s)	per region (ms)	D→H copy (s)	host side (s)
base	66.17	11.80	0.197	11.85
gp	28.25	5.04	0.191	11.63
Δ	−57.3% (2.34×)	—	−3%	−2%

4.3. The hybrid: rebalance, lane structure, binding resource

Steady-state means from the per-run stderr logs (`logs/{base,gp}.hybrid.r{2,3}.stderr`):

Tabla 4: Hybrid steady-state accounting. The GPU thread's "host side" is wall minus kernel minus D→H. "Busiest CPU worker" is the largest per-thread compute total among the 11 CPU workers — in both binaries it sits within 3% of the wall: the pool binds before and after.

	base	gp
wall (s)	29.83	23.72
GPU regions (of 5,608)	1,008 (18.0%)	3,006 (53.6%)
GPU kernel total (s)	25.31	10.69
GPU kernel avg (ms/region)	25.10	3.56
GPU kernel duty (% of wall)	84.9%	45.1%
GPU thread host side (s)	4.42	12.77
D→H copy, real (s)	0.096	0.258
CPU-pool regions	4,600	2,602
CPU-pool compute (s)	311.6	249.6
busiest CPU worker (s)	29.04	23.40

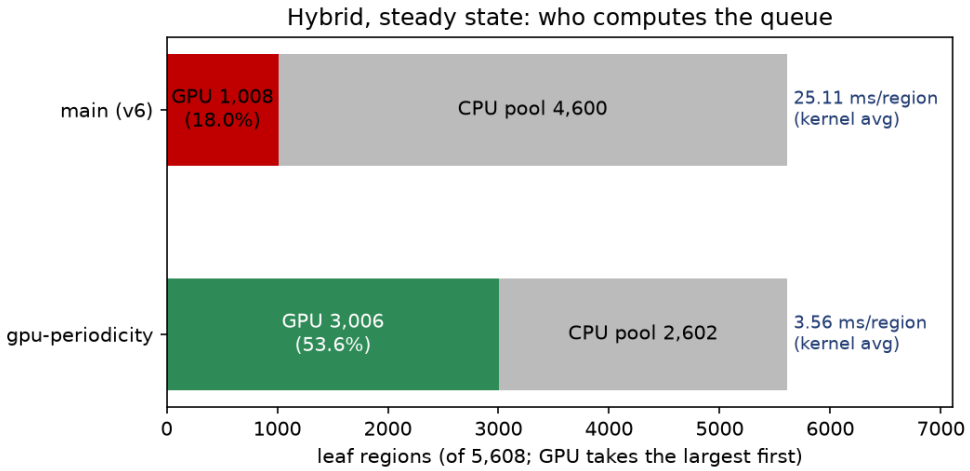

Figura 2: Who computes the queue (hybrid, steady state). The GPU's share of the 5,608 leaves triples from 18% to 54% — the min-max affinity queue re-routes automatically as the kernel gets cheap — while its average kernel time per region falls 25.10 → 3.56 ms.

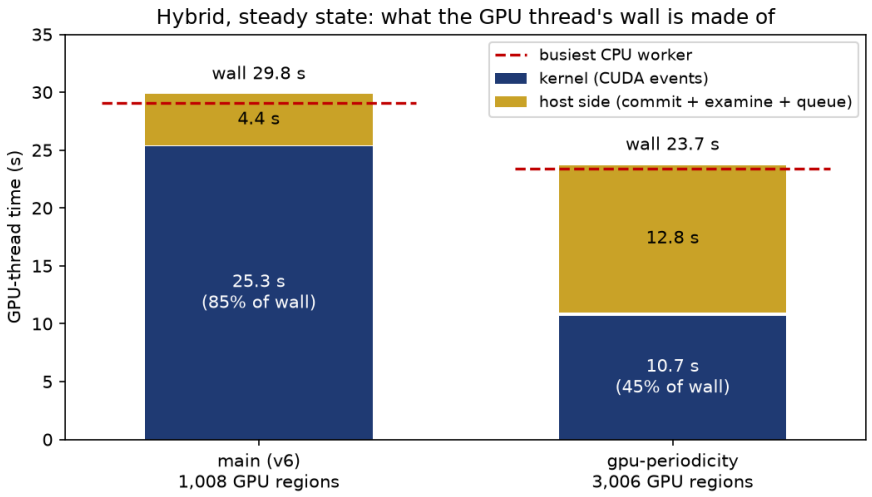

Figura 3: What the GPU thread's wall is made of (hybrid, steady state). Blue is kernel execution (CUDA events); gold is everything the thread does on the host — per-pixel commit, sampling, queue. The dashed red line is the busiest CPU worker: both resources finish together in both binaries. The check inverts the lane's cost structure — kernel occupancy falls from 85% to 45% of the wall, and the host side is now the lane's majority cost.

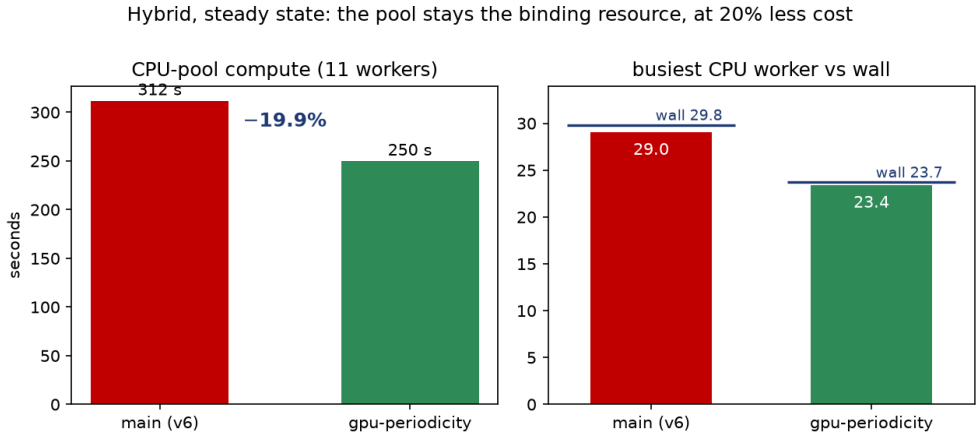

Figura 4: The transfer mechanism (hybrid, steady state). Left: total CPU-pool compute falls 20% because the GPU absorbs 2,000 regions the pool used to own. Right: the busiest CPU worker tracks the wall in both binaries — the pool is the binding resource before and after, so the pool relief is the wall win.

4.4. Output verification

`verify_identity.sh` renders the full production spec GPU-only with both binaries and byte-compares every frame: 100/100 byte-identical (mode-0 `cmp`). Since every region of every class goes through the modified kernel in that config, this simultaneously confirms the exactness argument on GPU arithmetic and the codegen precondition

(`nvcc` contracted the unchanged iteration statements identically in both builds). The decomposition is likewise unchanged: 5,608 leaves in every run of the batch.

5. Analysis

5.1. The "expect a loss" prediction is falsified, and the error is instructive

Mix-controlled kernel time fell 2.34× (Table 3); nothing about the result is marginal. The prediction failed because it priced the check with the wrong cost model. Report 16's 100.0% warp execution efficiency on interior regions was read as an asset the check would destroy — but on interior content that metric measured the waste itself: every lane of every warp executing all 10,000 iterations in lockstep is precisely what "100% efficiency, zero early exits" means. The check trades metric efficiency for eliminated work: a lane that detects its cycle at iteration ∼900 sits masked for the remainder of the warp's execution, and masked-idle costs nothing, whereas the "efficient" alternative was 9,000 further iterations of provably useless FP64. The warp pays the maximum of its lanes' detection latencies, and because a warp's 32 lanes are spatially contiguous pixels attracted to the same cycle, that maximum is a small multiple of the mean — not `MAXITER`. The implication generalises: warp execution efficiency is a throughput metric, valid only when the work being kept coherent is useful; reports 16/17's 100% baselines are hereby stale by design (recommendation 3).

5.2. Why 2.34× and not the CPU's 4.8×: the warp pays the max, and every lane pays the compares

The same check cut the CPU pool's compute 4.77× on this workload (1,917 → 402 s, report 21) but the kernel only 2.34×. The gap has two mechanisms, both structural. First, granularity: a CPU thread exits each pixel at that pixel's own detection latency, while a warp runs every pixel to the slowest of its 32 lanes — the max of 32 correlated latencies still exceeds their mean. Second, overhead incidence: the two FP64 compares are paid per iteration by every lane — including the escaping boundary pixels that never benefit — on a kernel that report 16 measured at 85.6% of the FP64 pipe. The check makes the per-iteration recurrence ∼25–30% more expensive and then wins anyway, because it deletes 80–90% of the iterations on interior content. Both mechanisms are intrinsic to SIMT early-exit; 2.34× is what "expect a loss" actually looks like when measured.

5.3. The hybrid self-rebalances a third time; the pool binds at 20% less cost

The wall win is a clean two-step transfer, and both steps are in Table 4. Step one: the kernel gets 7× cheaper per region (25.10 → 3.56 ms, partly mix — the GPU's marginal regions are now smaller), so the min-max affinity queue of report 12 automatically hands the GPU 3,006 regions instead of 1,008 (Fig. 2). This is the third self-rebalance in the series: v5's GPU held ∼61% of the queue, v6's CPU-side check pulled it down to ∼18–24% (the CPUs got fast, the GPU didn't), and v7 restores ∼54% now that both sides skip the grind. Step two: the 2,000 migrated regions carry 62 s of CPU-pool

compute (311.6 → 249.6 s, −19.9%), and since the busiest CPU worker tracks the wall in both binaries (within 3%, Fig. 4), the pool relief converts one-for-one into wall: −20.5%. The load balance itself never degrades — the pool's 11 workers stay within 3% of each other — which is the report-13 result surviving its third redistribution.

5.4. The GPU lane's cost structure inverts: the next lever is host-side commit, and it now transfers to wall

In the v6 hybrid the GPU thread spent 84.9% of the wall inside kernels; in v7 that is 45.1%, and the lane's majority cost is now its own host side: 12.77 s of per-pixel `setPixel` commit, stencil sampling and queue work against 10.69 s of kernel (Fig. 3). Real D→H transfer stays negligible (0.26 s total, ∼86 µs/region — report 15's finding, re-confirmed). Two facts make this actionable. First, the host cost is per-region invariant across binaries (4.4 and 4.3 ms/region) — it is commit work proportional to pixels, untouched by the check, exactly as expected. Second, the lane and the pool now finish together (both ≈ wall), so freeing lane time is no longer cosmetic: a cheaper commit lets the GPU pull further regions off a pool that is still the binding resource. Report 20's `scanLine` row-copy lever — replace the per-pixel `QImage::setPixel` loop with row-wise writes — was priced on the iGPU branch as a lane optimisation; on v7 it graduates to the top of the wall-relevant list, with ∼12.8 of 23.7 s as its theatre of operations (bounded, of course, by how much of that the pool can absorb).

5.5. Repricing, round three: the dGPU doubles its worker-equivalence

With the same-batch CPU12 anchor (35.78 s steady state), the worker-equivalence of report 22, $k = 12\,(t_{\mathrm{CPU12}}/t_{\mathrm{config}}) - 11$, prices the dGPU at $k = 3.4$ CPU workers under v6 and $k = 7.1$ under v7 — the check more than doubles the marginal value of the device it runs on, the mirror image of v6 halving it. The consequence for the parked `feat/igpu-opencl` branch is that its wash result (report 22) is stale in both directions: the iGPU's OpenCL kernel can take the same 15-line check (raising its raw speed), while the twice-repriced dGPU now clears the interior content the iGPU was queuing for (lowering the relief it can sell to the still-binding pool). Which force wins is a measurement (recommendation 4), not a derivation — report 21 teaches what trusting the derivation costs.

6. Recommendations

1. Merge `feat/gpu-periodicity` to `main`; tag `binary-v7-gpu-periodicity`. Identity gate 100/100, control clean at steady state (+0.07% in rep 3), hybrid −20.5% (23.72 s, best recorded), GPU-only −48.8%. Known trade-off: two FP64 compares per iteration are pure overhead on exterior-only content — bounded at the ∼25–30% per-iteration figure, and report 18 showed uniform-exterior is this workload's cheap case; accept it, as report 21 did for the CPU.

2. Cheapen the GPU thread's commit path (`scanLine` row copies instead of per-pixel `setPixel`). The lane's host side is now its majority cost (12.77 s vs 10.69 s kernel) and the pool it feeds still binds the wall, so lane time freed here converts

to wall until the two re-equalise. Expected shape: same two-step transfer as §5.3. Cost: a contained change in `mandelregion.cpp`'s GPU commit loop; A/B on its own branch.

3. Re-baseline the ncu warp metrics on v7. Reports 16/17 pinned interior kernels at 100.0% warp efficiency; the check spends that metric deliberately. One capture per the `experiments/16/17` scripts, so the number on record explains itself and nobody "fixes" the kernel back to efficient waste. Cost: one sudo capture session.

4. Re-price the iGPU on top of v7, after porting the check to `oclkernel.cpp`. Same 15-line mirror, same identity gate (per-device exactness holds for the OpenCL compiler's arithmetic too — byte-identity is only expected against its own baseline, as report 20 documented). Report 22's wash is stale in both directions (§5.5); one experiment-22-shaped sweep decides it.

5. Keep the cold-start discipline in future sweeps. The control caught a +6.4% rep-1 artefact that alternation alone did not remove (base runs first in every pair). Discard-first-rep — or a warm-up run — plus a null-config control should be standard for every future A/B on this machine; `ab.sh` already refuses to run off AC.

7. Conclusion

The GPU-side periodicity check does what report 21's recommendation said to measure and the opposite of what it said to expect: mix-controlled kernel time 2.34× (66.2 → 28.3 s over the identical 5,608 regions), GPU-only wall −48.8%, production hybrid 29.83 → 23.72 s (−20.5%, steady state, cold-start rep excluded by a CPU12 control that converges to +0.07%) — the best wall this project has recorded, with all 100 frames byte-identical and the 5,608-leaf decomposition unchanged. The prediction failed on its cost model: a warp pays its slowest lane's detection latency, not lockstep `MAXITER`, and 100% warp execution efficiency on interior content was a measurement of the waste, not of health. The hybrid self-rebalances a third time (GPU share 18 → 54%), the CPU pool remains the binding resource at 20% less cost, and the GPU thread's cost structure inverts — kernel occupancy 85 → 45% of wall — promoting host-side commit to the next lever. Open items: the 3-rep confirmation is done, so the merge is unblocked; the `scanLine` commit path, the ncu re-baseline, and the third iGPU repricing are queued behind it.